NATIONAL INSTITUTE OF TECHNOLOGY HAMIRPUR
Department of Computer Science and Engineering
Hamirpur (Himachal Pradesh) - 177005, India

Smart Tube: An IoT-Based Embedded Platform for Real-Time Water Flow and Temperature Control with Cloud-Integrated Monitoring

Dual Degree, CS - 744, Internet of Things, Project Report

Submitted By
Rishabh Raj (22dcs020)

Under the Guidance of
Dr. Robin Singh Bhadoria, Assistant Professor, Department of CSE

March 2026

Contents

Abstract

Domestic water dispensing systems are widely used but rarely designed with closed-loop intelligence, user safety automation, or remote telemetry. This project presents **Smart Tube**, an Internet of Things enabled embedded platform that regulates mixed water temperature and outlet flow in real time using an ESP32 controller, a PI control loop, and cloud-connected monitoring. The system reads water temperature using a DHT22 sensor and captures user intent through two potentiometers that set target temperature and desired flow opening. Two servos emulate mechanical valve actuation: one for a 3-way temperature mixing valve and one for a 2-way flow valve.

The firmware implements non-blocking scheduling through `millis()` timers and separates fast control updates from slower cloud transmission. Every control interval, the controller computes error between target and measured temperature and adjusts the mixing valve angle using proportional and integral terms. A dedicated anti-scald layer overrides all routine behavior and closes flow if sensed temperature crosses a safety threshold of 50 deg C. Local operator awareness is provided by a 16x2 I2C LCD that reports current mode, measured values, and actuator positions. Cloud telemetry is sent to ThingSpeak via HTTP at 15-second intervals with state-aware payload fields.

The platform was validated on the Wokwi simulation environment with repeatable response to setpoint changes, deterministic safety action under thermal excursions, and stable data upload cadence. The resulting prototype demonstrates that low-cost IoT control can reduce manual water adjustment effort, improve thermal safety, and provide actionable usage visibility for households and institutional facilities.

Keywords: IoT, ESP32, Smart Faucet, PI Controller, ThingSpeak, Embedded Control, Water Safety, Cloud Telemetry, Anti-Scald Protection, Real-Time Monitoring.

I. Introduction

Water use at taps and showers appears simple from a user perspective, but internally it is a dynamic control problem shaped by changing inlet temperatures, pressure fluctuations, and user comfort expectations. In conventional systems, users repeatedly adjust handles until acceptable temperature is reached, which causes avoidable water and energy waste. The lack of sensing and control also means that hazardous temperature spikes can reach the outlet without warning, creating safety risk for children and elderly users.

With the growth of low-cost microcontrollers and wireless networking, embedded intelligence can now be integrated into routine domestic utilities. The Internet of Things provides a practical framework for this integration by combining local sensing, edge decision logic, actuation, and cloud analytics. In the Smart Tube project, this paradigm is applied to a water faucet use case where both temperature and flow are controlled automatically while remaining configurable through direct user input.

The proposed system is intentionally designed as a deployable learning bridge between simulation and real-world engineering. Hardware-level control concepts such as sensor acquisition, PWM-based actuation, and fail-safe interlocks are combined with software concepts including non-blocking scheduling, state machines, and cloud APIs. This combination makes the project relevant not only as a prototype utility device but also as a compact IoT systems case study.

From a societal perspective, the project aligns with three practical goals. First, it improves thermal safety through deterministic anti-scald logic. Second, it reduces repeated manual adjustment and associated wastage. Third, it introduces visibility through cloud logging so that long-term behavior can be inspected rather than assumed.

II. Problem Statement

The Smart Tube system addresses the following four critical problems observed in traditional faucet operation.

P1 - Unsafe Outlet Temperature Events: Conventional fixtures do not provide automatic thermal cut-off. If hot supply temperature suddenly rises, users may be exposed to outlet temperatures beyond safe skin contact levels.

P2 - Manual Trial-and-Error Mixing: Without closed-loop control, users repeatedly adjust hot and cold proportions to reach comfort temperature. This introduces delay, discomfort, and unnecessary water consumption during settling.

P3 - Lack of Adaptive Temperature Stabilization: When inlet conditions drift because of geyser cycling or line pressure variations, manually set valves cannot self-correct. Output temperature oscillates unless the user continuously intervenes.

P4 - No Remote Operational Visibility: Households and facility managers generally cannot inspect outlet behavior over time. Abnormal states, over-temperature episodes, or unusual usage patterns remain undocumented.

Project Objectives

To solve the above issues, the project defines five implementation objectives:

1. Measure mixed water temperature continuously using an embedded sensor channel.
2. Accept user-defined setpoints for both temperature and flow.
3. Apply PI-based closed-loop control to maintain target temperature.
4. Enforce immediate anti-scald override at or above 50 deg C.
5. Upload periodic telemetry to cloud infrastructure for monitoring and retrospective analysis.

These objectives convert a purely manual utility interaction into a measurable and controllable cyber-physical system suitable for future scaling.

To solve the above issues, the project defines five implementation objectives:

1. Measure mixed water temperature continuously using an embedded sensor channel.
2. Accept user-defined setpoints for both temperature and flow.
3. Apply PI-based closed-loop control to maintain target temperature.
4. Enforce immediate anti-scald override at or above 50 deg C.
5. Upload periodic telemetry to cloud infrastructure for monitoring and retrospective analysis.

These objectives convert a purely manual utility interaction into a measurable and controllable cyber-physical system suitable for future scaling.

III. System Architecture and Design

A. Hardware Architecture Overview

The architecture follows a layered IoT model with clear separation between perception, processing, actuation, and communication layers.

At the perception layer, two analog potentiometers capture user intent: one maps to target temperature and one maps to desired flow opening. A DHT22 channel provides measured temperature feedback used by the control loop. At the processing layer, the ESP32 performs ADC sampling, control computation, state evaluation, and telemetry assembly. At the actuation layer, two servos represent valve mechanisms. Servo-Temp drives the 3-way mixing path and Servo-Flow drives the outlet opening position. At the communication and presentation layer, a 16x2 LCD delivers local status while Wi-Fi transport pushes cloud fields to ThingSpeak.

The model supports deterministic local behavior even when cloud services are unavailable because core control decisions are edge-resident. This is essential for safety-oriented systems where remote connectivity cannot be the primary dependency.

B. Functional Block Description

1. **Input Conditioning Block:** acquires ADC channels and maps raw values to engineering ranges.
2. **Thermal Feedback Block:** reads temperature sensor and validates numeric integrity.
3. **Control Block:** computes PI output and applies actuator saturation limits.
4. **Safety Block:** evaluates critical threshold and overrides all non-emergency logic.
5. **Display Block:** renders current state and live measurements for user awareness.
6. **Cloud Block:** dispatches state variables at API-compliant intervals.

Together, these blocks establish a robust control pipeline from user intent to physical action and recorded telemetry.

III. System Architecture and Design (Continued)

C. Software Architecture and Scheduling

Firmware organization uses cooperative multitasking through non-blocking time checks. Instead of `delay()` driven loops, each subsystem is executed when its timer window opens. This ensures that cloud communication latency never stalls local actuation.

Two main periodic tasks are used:

- **Control task (2 s):** read sensors, compute PI command, evaluate safety, update actuators, refresh LCD.
- **Cloud task (15 s):** package selected fields and perform HTTP upload to ThingSpeak.

The execution flow can be summarized as:

1. Acquire setpoints and measured temperature.
2. Validate sensor output; skip invalid samples.
3. Run safety check for thermal threshold crossing.
4. If safe, execute PI update and actuator write.
5. Update local state and display content.
6. On telemetry interval, publish state fields to cloud.

D. Control Formulation

Let target temperature be T_{set}, measured temperature be T_{meas}, and error be:

$$e(k) = T_{set}(k) - T_{meas}(k)$$

The discrete PI signal is:

$$u(k) = K_p \cdot e(k) + K_i \cdot \int e$$

with controller constants selected in the implementation as $K_p = 4.0$, $K_i = 0.5$, and control step $DT = 2$ s. Output saturation is enforced in actuator range $[0, 180]$. Anti-windup logic updates the integral term only when unsaturated control is within bounds, preventing runaway accumulation during saturation episodes.

This design keeps the control law simple, interpretable, and stable for the operating range used in simulation.

IV. Component Descriptions and Characteristics

A. ESP32 DevKit V1 - Central Controller

The ESP32 acts as the computational core and communication endpoint. It provides sufficient GPIO, ADC channels, PWM generation, and integrated Wi-Fi for complete edge execution without external communication modules. Its memory and clock resources are adequate for task scheduling, local control, and HTTP telemetry while maintaining real-time responsiveness.

B. DHT22 Temperature Sensor

The DHT22 channel is used as mixed water temperature feedback in the simulation model. Although physical deployments may prefer waterproof probes, the DHT22 is suitable for validating acquisition, filtering, and control response in a lab-scale prototype. Its accuracy and stability make it useful for controller testing when interpreted with proper guard logic.

C. Potentiometers for User Setpoints

Two potentiometers represent direct user interface knobs. P1 maps to target temperature in the range 20 deg C to 45 deg C, and P2 maps to flow angle in the range 0 deg to 180 deg. The mapping strategy limits temperature input to a safe domestic comfort envelope while preserving flexible flow-domain experimentation.

D. Servo Actuators for Valve Emulation

Two hobby-class servo motors emulate mechanical valves. Servo-Temp controls the 3-way mixer equivalent, and Servo-Flow controls outlet opening equivalent. In each control cycle, Servo-Temp receives the PI-computed angle while Servo-Flow receives the user flow setpoint, which separates comfort regulation from flow preference.

IV. Component Descriptions and Characteristics (Continued)

E. LCD 16x2 I2C Display

The display provides immediate local observability through mode-specific messages. NORMAL shows measured temperature, setpoint, and both servo angles. FLOW CLOSED indicates zero or near-zero flow, and EMERGENCY indicates that thermal override is active. Local visualization supports usability and debugging during tuning sessions.

F. Connectivity and Validation Parameters

The firmware relies on a compact library set: `WiFi.h` for network association, `HTTPClient.h` for cloud upload, `ESP32Servo.h` for servo control, `LiquidCrystal_I2C.h` and `Wire.h` for display transport, and `DHT.h` for sensor sampling. The main validation parameters are cold source 15 deg C, hot source 65 deg C, critical threshold 50 deg C, control sampling interval 2 s, and cloud update interval 15 s.

V. Implementation

A. Sensor Acquisition and Input Mapping

At each control step, ADC channels are sampled and converted into operating setpoints. Flow setpoint is linearly mapped from raw ADC to servo domain $[0, 180]$. Temperature setpoint is mapped from raw ADC to $[20, 45]$ deg C. This dual mapping provides intuitive user control while preserving safety boundaries.

Temperature feedback is read from the DHT22 sensor. If an invalid sample (`NaN`) is detected, the control step is skipped to prevent unstable or undefined control commands. This simple guard significantly improves runtime robustness.

B. PI Control Engine

The controller computes thermal error and combines proportional and integral actions. Proportional response quickly corrects immediate deviation, while the integral term removes persistent offset caused by model mismatch or slow thermal drift.

Implementation sequence:

1. Compute current error $e = T_{set} - T_{meas}$.
2. Predict unsaturated control using tentative integral update.
3. Apply anti-windup rule: integrate only when control remains inside bounds.
4. Recompute control with approved integral state.
5. Constrain final command in $[0, 180]$ and write to Servo-Temp.

This sequence avoids integral runaway when actuator limits are reached and improves settling consistency after large transients.

C. Flow Control Coupling

The flow channel is intentionally user-priority driven. Servo-Flow directly tracks mapped flow setpoint so users can control outlet openness independent of temperature dynamics, except during emergency override conditions.

V. Implementation (Continued)

D. Safety Override and State Management

Safety logic executes before normal actuator updates. If measured temperature crosses $T_{critical} = 50$ deg C, the firmware triggers immediate override behavior:

1. Set flow actuator to closed position.
2. Move temperature mixing actuator toward cold side.
3. Reset integral accumulator to avoid rebound on recovery.
4. Publish emergency state to display and telemetry fields.

This deterministic branch has strict priority over comfort control and remains the key safety guarantee in the system.

E. Display and Human Feedback

The LCD is updated only when needed to reduce flicker and retain readability. State transitions trigger a clear screen event; otherwise, values are refreshed in-place. This small implementation choice improves perceived quality during continuous operation.

F. Cloud Telemetry Pipeline

A periodic uploader sends key fields to ThingSpeak every 15 seconds to comply with API timing limits. Payload fields include measured temperature, target temperature, flow setpoint, PI output angle, and operating state. If Wi-Fi is unavailable, reconnection is attempted before publish. Local control continues regardless of temporary cloud failure.

G. Implementation Integrity Checks

The project includes practical safeguards that make the firmware stable for long runtime windows:

- timer-based task separation,
- constrained actuator ranges,
- invalid sensor sample filtering,
- emergency-first control branching,
- bounded cloud publish cadence.

These checks collectively convert a classroom demonstration into a disciplined embedded control prototype.

VI. Results and Discussion

A. Functional Validation Summary

Validation was performed on Wokwi using repeated scenario sweeps over temperature and flow inputs. The system consistently executed all major functions: setpoint acquisition, PI correction, actuator updates, LCD state rendering, and periodic cloud upload.

B. Control Behavior Under Setpoint Changes

When target temperature was changed through P1, the mixing servo responded in the expected direction and converged without sustained oscillation under selected gains. The anti-windup rule prevented integral growth during saturated commands, which reduced overshoot after large adjustments. Under steady conditions, measured temperature tracked setpoint with acceptable error band for prototype-grade sensing.

C. State Transition Behavior

Three states were observed and verified:

1. **NORMAL:** closed-loop temperature and user flow control active.
2. **FLOW CLOSED:** user set flow to near-zero, with safe idle behavior.
3. **EMERGENCY:** thermal threshold exceeded and automatic shutoff engaged.

Transitions were deterministic and reversible when conditions returned to safe ranges.

D. Reliability of Execution Timing

The separation of 2-second control and 15-second upload loops prevented cloud operations from disturbing local control cadence. This was especially useful during repeated dashboard updates where communication jitter could otherwise affect actuator timing.

These observations indicate that architecture-level scheduling choices were as important as control-law tuning for stable perceived behavior.

VI. Results and Discussion (Continued)

E. Safety Response Evaluation

Scald-protection behavior was evaluated by forcing sensor readings at and above the critical threshold. In all tested cases, flow closure and emergency messaging occurred immediately in the next control cycle. Integral reset prevented delayed high-angle rebound once emergency state cleared.

F. Cloud Logging and Data Visibility

ThingSpeak accepted periodic HTTP updates with stable cadence at 15-second intervals. The selected fields were sufficient to reconstruct key runtime behavior remotely:

- thermal trend (T_{meas}),
- operator intent (T_{set} and flow setpoint),
- controller action (mixing angle),
- finite state status.

This data foundation enables future diagnostics such as identifying repeated emergency events or prolonged high-energy operating profiles.

G. Cost and Deployment Feasibility

Prototype hardware remains low-cost and accessible for student and household experimentation. The low entry cost supports iterative refinement without high replacement risk. At the same time, transition to production hardware will require stronger actuators, waterproof sensors, and plumbing-grade mechanical coupling.

H. Limitations

The current validation is simulation-centered and does not include pressurized water-line mechanics, valve torque under real flow, waterproofing constraints, or long-term calibration drift. Therefore, results should be interpreted as a functionally verified proof-of-concept, not final field certification.

VII. Future Integration and Large-Scale Applications

A. Hardware Hardening for Real Installations

A production-focused Smart Tube should replace hobby servos with industrial-grade proportional valves and include torque-characterized actuation modules designed for actual pressure ranges. Temperature sensing should migrate to waterproof probes suitable for contact with flowing water. Enclosure design must include ingress protection and electrically isolated power domains for wet environments.

B. Advanced Control and Adaptive Tuning

While PI control is effective for this prototype, future versions can incorporate gain scheduling, disturbance observers, or lightweight model-predictive logic to improve response under rapidly varying inlet conditions. Auto-tuning workflows can be added so installation technicians can calibrate behavior for specific plumbing layouts.

C. Mobile and Voice Interfaces

A companion mobile interface can expose user profiles such as preferred morning and evening temperature presets. Voice integration through common assistants can allow hands-free operation, while secure local fallback should remain available to prevent lockout during cloud outages.

D. Telemetry at Building Scale

For hotels, hostels, and hospitals, fleet-level deployment can aggregate many Smart Tube nodes to a centralized dashboard. Cross-unit analytics can identify abnormal consumption, detect repeated safety events, and prioritize maintenance.

This roadmap demonstrates that the current platform can evolve from a single-node prototype into a practical component of smart facility infrastructure.

VIII. Conclusion

This report presented Smart Tube, an IoT-enabled embedded water-control platform designed to improve temperature safety, reduce manual adjustment effort, and provide cloud-visible operation. The project integrates sensing, edge control, actuation, state management, and telemetry in one cohesive architecture suitable for course-level demonstration and future engineering extension.

Non-blocking scheduling separated control and cloud responsibilities into predictable timing windows, ensuring that network operations did not compromise local safety or responsiveness. This architecture decision, combined with explicit state encoding (`NORMAL`, `FLOW CLOSED`, `EMERGENCY`), improved interpretability for both operators and remote observers.

The work confirms that a low-cost ESP32-based design can deliver meaningful safety and usability gains in domestic water systems. Although deployment-grade upgrades are required for real plumbing conditions, the prototype establishes a validated technical foundation for smarter, safer, and more observable water delivery systems.

In summary, Smart Tube demonstrates how IoT and embedded control can convert a routine utility interaction into a measurable cyber-physical workflow with clear user and operational benefits.

References

[1] Bureau of Energy Efficiency, Government of India, "Energy Efficiency Trends in Domestic Water Heating," Annual Report, New Delhi, 2024.

[2] World Health Organization, "Burn Prevention and Scald Injury Factsheet," WHO Technical Brief, Geneva, 2023.

[3] Espressif Systems, "ESP32 Technical Reference Manual," Version 4.9, 2024. Available: https://docs.espressif.com

[4] Adafruit Industries, "DHT22 Sensor Guide and Electrical Characteristics," 2023. Available: https://learn.adafruit.com

[5] MathWorks, "ThingSpeak Documentation: REST API and Channel Fields," 2025. Available: https://thingspeak.com/docs

[6] OASIS Standard, "MQTT Version 5.0," 2019. Available: https://docs.oasis-open.org/mqtt

[7] A. McEwen and H. Cassimally, *Designing the Internet of Things*, Wiley, 2014.

[8] K. J. Astrom and T. Hagglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed., ISA, 1995.

[9] S. Monk, *Programming the ESP32 in Arduino*, 2nd ed., McGraw Hill, 2022.

[10] N. O'Leary, "PubSubClient MQTT Library," GitHub Repository, 2025. Available: https://github.com/knolleary/pubsubclient

[11] B. Adams, "DHTesp Library for ESP32/ESP8266," GitHub Repository, 2025. Available: https://github.com/beegee-tokyo/DHTesp

[12] Wokwi Online Simulator, "ESP32 Simulation Documentation," 2025. Available: https://wokwi.com

[13] International Energy Agency, "Digitalization and Energy End-Use Systems," IEA Insights, 2023.

[14] Ministry of Housing and Urban Affairs, Government of India, "Urban Water Use Efficiency Guidelines," Policy Note, 2024.